

File Read/Write

1. Explain the difference between opening a file in **'w' (write) mode** versus **'a' (append) mode**. What happens to the existing content of the file in each case?
 2. Why is it considered a best practice to use the **with statement** (e.g., with `open('file.txt', 'r')` as `f:`) when working with files? What critical problem does it solve automatically?
 3. What is the role of the **file cursor** (or file pointer)? How do the `seek()` and `tell()` methods allow you to interact with it?
 4. Differentiate between reading a file with `read()`, `readline()`, and `readlines()`. In what situation would you prefer one over the others, especially considering memory usage with large files?
 5. What is the conceptual difference between opening a file in **text mode** (e.g., `'r'`) versus **binary mode** (e.g., `'rb'`)? Give an example of a file type you must open in binary mode and explain why.
 6. Imagine you are writing a large list of strings to a file. Is it more memory-efficient to first join all strings into one giant string and write it once, or to loop through the list and write each string individually? Explain your reasoning.
 7. What is **buffering** in the context of file I/O? How can it affect the performance of your program and the integrity of your data if the program crashes unexpectedly?
 8. When handling files, you often need to interact with the operating system itself. What is the purpose of Python's **os module** in this context? Name two common file-related tasks you would use it for.
 9. Why is it better to use a specialized module like **csv** or **json** to read/write those file types instead of just treating them as plain text files with `read()` and `write()`?
 10. What potential **Exceptions** should you be prepared to handle when working with files? Describe at least two common ones (e.g., `FileNotFoundError`, `PermissionError`) and the circumstances that cause them.
-

Debugging

1. What is the conceptual difference between a **SyntaxError** and an **Exception** (like a `NameError` or `TypeError`)? At what point in a program's execution does each one occur?
2. Explain the purpose of a **traceback**. What are the two most important pieces of information you can get from it to find a bug?

3. Describe the "print debugging" technique. What are its main advantages and disadvantages compared to using a dedicated **debugger tool**?
 4. What is a **breakpoint**? In the context of a debugger like pdb, what happens to your program's execution when it hits a breakpoint?
 5. Explain the difference between "**stepping over**," "**stepping into**," and "**stepping out**" when you are paused at a line of code in a debugger. When would you use each action?
 6. Why is it important for code to "**fail-fast**"? How does this principle make debugging easier in the long run?
 7. Explain the purpose of a **try...except** block. How can it make your program more robust? Conversely, how can a poorly written except block (e.g., `except Exception: pass`) make debugging a nightmare?
 8. What is "**rubber duck debugging**"? Explain the psychological principle that makes this simple, offline technique surprisingly effective. 🦆
 9. Describe the concept of **logging**. How is using the logging module fundamentally different and often better than using `print()` statements for debugging, especially in a large or long-running application?
 10. If you encounter a bug that you've never seen before, what is a systematic approach to narrowing down its cause? (Hint: Think about isolating the problem).
-

Classes and Object Methods

1. What is the fundamental difference between a **class** and an **object** (or instance)? Use a real-world analogy like a blueprint and a house.
2. Explain the purpose of the `__init__` method. Why is it called a "constructor," and when is it automatically called?
3. What does the `self` parameter represent inside an instance method? Why must it always be the first parameter?
4. Differentiate between an **instance attribute** and a **class attribute**. How do they differ in terms of scope and how they are shared among objects of the same class?
5. What are the key differences between an **instance method**, a **class method** (using `@classmethod`), and a **static method** (using `@staticmethod`)? Explain what their first argument is (`self`, `cls`, or nothing).
6. In what scenario would you choose to use a `@classmethod`? Provide a practical example, such as creating an object from an alternative data source (e.g., a CSV string).
7. When is a `@staticmethod` useful? Why would you place it inside a class if it doesn't operate on the instance or the class itself?

8. What are "**dunder**" (double underscore) methods like `__str__` and `__repr__`? What is their primary purpose and how do they allow your custom objects to integrate with built-in Python functions?
 9. Explain the concept of an object's **state**. How do an object's attributes and methods work together to define and manage its state throughout its life?
 10. Can you have an object without first defining a class in Python? Explain why or why not, relating your answer back to Python's data model where everything is an object.
-

Inheritance

1. What is **inheritance** in object-oriented programming? Explain the concept using the "is-a" relationship (e.g., "A Car is a Vehicle").
 2. What are the two primary benefits of using inheritance in your code? (Hint: Think about code reuse and logical structure).
 3. Define the terms **superclass** (or base class) and **subclass** (or derived class). How is this parent-child relationship established in Python's syntax?
 4. What happens if a subclass defines a method with the exact same name as a method in its superclass? What is this concept called?
 5. What is the purpose of the `super()` function? Provide a critical scenario where you *must* use `super()` inside a subclass's `__init__` method.
 6. Explain the concept of **Method Resolution Order (MRO)**. Why is it crucial in languages that support **multiple inheritance**?
 7. Briefly describe the "**diamond problem**" in multiple inheritance. How does Python's MRO prevent ambiguity and solve this problem?
 8. Differentiate between an "**is-a**" relationship and a "**has-a**" relationship. Which one is modeled by inheritance, and which one is modeled by **composition** (using other objects as attributes)?
 9. Give an example of a "has-a" relationship. For instance, in modeling a Car and an Engine, why is composition a better choice than having Car inherit from Engine?
 10. Can a subclass access methods and attributes from its "grandparent" class? Explain how the chain of inheritance works.
-

Polymorphism

1. In simple terms, what is **polymorphism**? The name means "many forms" — how does this translate to the behavior of Python functions and methods?
 2. Explain how Python's **duck typing** embodies the principle of polymorphism. What does the phrase "If it walks like a duck and quacks like a duck, it's a duck" mean in a coding context? 🦆
 3. How does **method overriding** (from inheritance) enable polymorphism? Provide a simple example with a `Animal` superclass and `Dog` and `Cat` subclasses, each with a `speak()` method.
 4. Imagine a function `calculate_area(shape)` that works correctly with `Circle` objects and `Square` objects. For this to be possible, what must be true about the `Circle` and `Square` classes?
 5. What is **operator overloading**? How is it a form of polymorphism? Give an example using a dunder method like `__add__` to allow the `+` operator to work on your custom objects.
 6. Can polymorphism exist in Python **without using inheritance**? Explain your answer with an example.
 7. What is the main benefit of writing polymorphic code? How does it make your functions more **flexible and extensible**?
 8. Differentiate between **run-time (or dynamic) polymorphism** and **compile-time (or static) polymorphism**. Which type is characteristic of Python and why?
 9. Imagine you have a list containing different objects: `[FileLogger(), ConsoleLogger(), DatabaseLogger()]`. If you loop through this list and call `log.write(message)` on each object, this is polymorphism. What is the key requirement for all these logger classes for this code to work?
 10. How do **Abstract Base Classes (ABCs)** from the `abc` module help enforce a polymorphic interface? What do they guarantee about any subclass that inherits from them?
-

Building Class Hierarchies

1. When designing a class hierarchy, what is the goal of the **top-level base class**? What kind of general attributes and methods should it contain?
2. Explain the principle of moving **common functionality "up" the hierarchy**. If `Dog` and `Cat` classes both have a `name` attribute and an `eat()` method, where should you define them to avoid code duplication?

3. What is the **Liskov Substitution Principle (LSP)**? In simple terms, how does it guide you to build a logical and reliable class hierarchy?
 4. Give a classic example where the Liskov Substitution Principle is violated, such as the **Square-Rectangle problem**. Why can't a Square always be substituted for a Rectangle without issues?
 5. When is it more appropriate to use **composition** (a "has-a" relationship) instead of **inheritance** (an "is-a" relationship)? Give an example.
 6. You need to model users in an application: Guest, Member, Moderator, and Admin. Sketch out a logical class hierarchy for these roles. Which class would be the base?
 7. What is a **"mixin" class**? How can mixins be used to add specific, reusable functionality (e.g., logging) to different classes in a hierarchy without being part of the primary "is-a" relationship?
 8. What is the difference between a **"deep"** and a **"shallow"** class hierarchy? What are the potential downsides of a very deep hierarchy (e.g., 8+ levels of inheritance)?
 9. Why might you design a class hierarchy where the base class is **never intended to be instantiated** on its own? What is this type of class called?
 10. You are modeling geometric shapes. How would you decide whether a Square should inherit from a Rectangle, or if they should both inherit from a more general Quadrilateral? What are the pros and cons of each approach?
-

Encapsulation & Abstraction

1. Explain the concept of **encapsulation**. Why is it described as "bundling data and the methods that operate on that data" into a single object?
2. How does encapsulation support **data hiding**? What is the main benefit of preventing direct, uncontrolled access to an object's internal state?
3. Python doesn't have a private keyword. What naming convention does it use to signal that an attribute or method is intended for **internal use only**?
4. What is the practical difference between a "protected" attribute (e.g., `_name`) and a "private" attribute (e.g., `__name`) in Python? What does Python's name mangling do to the `__` attribute?
5. Explain **abstraction**. How does it help manage complexity by hiding implementation details and showing only an essential, simplified interface? Use a real-world analogy like a car's dashboard. 🚗
6. How do encapsulation and abstraction work together? Can you have one without the other?

7. What is an **Abstract Base Class (ABC)** and how does it serve as a powerful tool for enforcing abstraction? What happens if a subclass fails to implement a required abstract method?
 8. Why is it better to use "getter" and "setter" methods (or properties) to manage an attribute instead of allowing direct public access?
 9. Explain the role of the **@property decorator**. How does it allow you to expose a public interface that looks like an attribute but behaves like a method, thus supporting encapsulation?
 10. For a `BankAccount` class, which details would you expose as part of its public interface (**abstraction**), and which details would you hide as part of its internal implementation (**encapsulation**)?
-

Real-World Modeling

1. You need to model a simple **library system**. What would be the three primary classes you'd create? What is the relationship between them (e.g., inheritance or composition)?
2. Imagine modeling an **e-commerce system**. How would you represent the relationship between a Customer, an Order, and a Product? Which class would be a container for the others?
3. You are modeling a social media platform. A User can create a Post. How would you model different types of posts like `TextPost`, `ImagePost`, and `VideoPost` using a class hierarchy? What common methods might they all share?
4. In a **video game**, you have different character types like Warrior, Mage, and Archer. What attributes would you put in a base Character class, and what attributes would be specific to the subclasses?
5. When modeling a Car, why is the Engine a prime candidate for **composition** (a Car *has an* Engine) rather than inheritance (a Car *is an* Engine)?
6. You're building a system to model a **pizza restaurant**. How would you use objects and classes to represent a customer's order, which includes a Pizza object that itself has multiple Topping objects?
7. How do you decide what should be an **attribute** versus what should be a **method**? For a Student class, would `date_of_birth` be an attribute or a method? What about `calculate_age()`?
8. When modeling a system, what is the danger of trying to model **every single detail** from the very beginning? Why is an iterative approach (starting simple and adding complexity) generally better?
9. Model a digital music player. What classes would you need for a Song, an Album, an Artist, and a Playlist? How would these classes interact with each other?

10. You need to model a **workflow system** where a Document goes through various states like Draft, InReview, Approved, and Rejected. How could a State design pattern help you model this, where the document's behavior changes depending on its current state?
-